

Chapter 2: Prerequisites & AWS Environment Setup

2.1 Installing AWS CLI, Terraform, and Ansible — Enterprise Smart Library Management Platform

Curriculum Module	Target Platform	Focus Tools	Release Status
Unit 4: Infrastructure Automation	Ubuntu 22.04 LTS / macOS / WSL2	AWS CLI v2, Terraform, Ansible	v1.0.0 Verified Lab Guide

1. Chapter 2 Overview & Objectives

Before executing Infrastructure as Code (IaC) or Configuration Management automation, engineers must establish a standardized, deterministic local workstation environment. This guide provides exact, production-verified instructions to install, configure, authenticate, and validate the three foundational tools required for the Enterprise Smart Library Management Platform:

- **AWS Command Line Interface (AWS CLI v2):** The unified tool to manage AWS cloud services, query resources, and authenticate programmatic API sessions.
- **HashiCorp Terraform (v1.7+):** The declarative Infrastructure as Code engine used to provision cloud fabric (VPCs, Subnets, Security Groups, EC2 compute, and S3 storage).
- **Ansible Core (v2.16+):** The agentless configuration management engine executing playbooks over OpenSSH to prepare servers, install Docker and Nginx, and launch the application.

LAB PREREQUISITES & SYSTEM REQUIREMENTS

Operating System: Linux (Ubuntu 20.04/22.04 LTS recommended), Debian 11/12, macOS (Apple Silicon or Intel), or Windows 10/11 running Windows Subsystem for Linux 2 (WSL2 Ubuntu).

Hardware: Minimum 2 CPU cores, 4 GB RAM, 15 GB free disk space, and outbound broadband internet connectivity on ports 80, 443, and 22.

2. Section 2.1: Installing AWS CLI, Terraform, and Ansible

2.1.1 Step 1: Workstation Baseline Utilities

Ensure local package repositories are refreshed and necessary archive utilities, compilers, and security certificates are installed:

Terminal: Updating Repositories and Installing Baseline Tools

```
# Update local package indexes and upgrade outdated packages
sudo apt-get update && sudo apt-get upgrade -y

# Install core system utilities, cURL, unzipping tools, and Python prerequisites
sudo apt-get install -y curl wget unzip jq git software-properties-common ca-certificates gnupg apt-transport-https python3 python3-pip python3-venv
```

2.1.2 Step 2: Installing AWS Command Line Interface (AWS CLI v2)

AWS CLI version 2 is the official modern binary distribution for managing Amazon Web Services. Unlike version 1, AWS CLI v2 is distributed as a self-contained bundle that does not rely on host Python environments.

Terminal: Downloading and Installing AWS CLI v2 on Linux (x86_64)

```
# 1. Download official AWS CLI v2 installer package
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

# 2. Extract the downloaded archive
unzip -q awscliv2.zip

# 3. Execute installation script (creates binaries in /usr/local/bin)
sudo ./aws/install --update

# 4. Clean up installation artifacts
rm -rf aws awscliv2.zip

# 5. Verify installation
aws --version
```

Note for macOS & Windows WSL2 Users

macOS (Homebrew): Run `brew install awscli terraform ansible`.

macOS (Official PKG): Download AWSCLIV2.pkg from Amazon's official documentation.

Windows WSL2: Follow the Linux terminal commands above inside your Ubuntu WSL2 bash shell.

2.1.3 Step 3: Installing HashiCorp Terraform CLI

HashiCorp signs all package releases using official GPG keys. We securely register HashiCorp's official APT repository and install the verified Terraform binary:

Terminal: Registering HashiCorp GPG Key and Installing Terraform

```
# 1. Add HashiCorp's official GPG signing key to secure keyrings
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-
archive-keyring.gpg

# 2. Add HashiCorp official Debian/Ubuntu repository source list
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://
apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list

# 3. Update APT package index to index HashiCorp repository
sudo apt-get update

# 4. Install Terraform CLI
sudo apt-get install -y terraform

# 5. Enable optional shell autocomplete for Bash / Zsh
terraform -install-autocomplete

# 6. Verify Terraform installation
terraform -version
```

2.1.4 Step 4: Installing Ansible Core & Dependencies

Ansible requires Python 3.10+ on the control node. On Ubuntu, installing from the official Ansible Personal Package Archive (PPA) ensures access to the latest stable release of Ansible Core and built-in modules:

Terminal: Adding Ansible PPA and Installing Ansible Core

```
# 1. Add Ansible official PPA repository
sudo add-apt-repository --yes --update ppa:ansible/ansible

# 2. Install Ansible Core package
sudo apt-get install -y ansible

# 3. Install community and AWS cloud collection extensions
ansible-galaxy collection install amazon.aws community.general

# 4. Verify Ansible installation and Python runtime path
ansible --version
```

3. Section 2.2: AWS Programmatic Authentication & STS Verification

To permit Terraform and AWS CLI to provision infrastructure, you must configure programmatic credentials using an IAM user with appropriate permissions (e.g., EC2, VPC, S3, and Security Group policies).

3.1 Configuring AWS Credentials via CLI

Terminal: Running 'aws configure' Interactive Setup

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wjaIrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json
```

3.2 Verifying AWS Security Token Service (STS)

Verify that your credentials are functioning and identify the authenticated IAM principal:

Terminal: Verifying Authenticated IAM Identity

```
aws sts get-caller-identity
```

Expected Terminal Output: JSON Caller Identity

```
{
  "UserId": "AIDAS6SAMPLEUSEREXAMPLE",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/devops-trainer-admin"
}
```

SECURITY GOVERNANCE NOTE

Never hardcode AWS Access Keys into Git repositories or Terraform `.tf` files. The AWS CLI stores credentials securely in `~/.aws/credentials` and `~/.aws/config`, which Terraform automatically detects and uses at runtime.

4. Section 2.3: SSH Key Pair Generation & EC2 Import

Ansible operates agentlessly over OpenSSH. To enable Ansible to authenticate securely with the provisioned EC2 instance, generate a 4096-bit RSA key pair on your local machine and import the public key directly to AWS:

4.1 Generating 4096-Bit RSA SSH Key Pair

Terminal: Generating Key Pair with Strict File Permissions

```
# 1. Create secure SSH directory if not present
mkdir -p ~/.ssh

# 2. Generate a 4096-bit RSA key without passphrase for automated CI/CD execution
ssh-keygen -t rsa -b 4096 -f ~/.ssh/library-aws-key -N "" -C "devops@careertiq-library"

# 3. Restrict private key permissions to read-only for owner (Mandatory for OpenSSH)
chmod 400 ~/.ssh/library-aws-key
chmod 644 ~/.ssh/library-aws-key.pub

# 4. Confirm key generation
ls -la ~/.ssh/library-aws-key*
```

4.2 Importing Public Key into AWS EC2 Key Pairs

Terminal: Registering Public Key in Region us-east-1

```
aws ec2 import-key-pair --key-name "library-aws-key" --public-key-material fileb://~/.ssh/library-aws-key.pub --region us-east-1
```

Expected Output: AWS EC2 Key Pair Confirmation

```
{
  "KeyFingerprint": "b4:37:38:12:44:98:ef:5a:7b:19:30:fa:8c:11:7d:e2",
  "KeyName": "library-aws-key",
  "KeyPairId": "key-0a9b8c7d6e5f41234"
}
```

5. Section 2.4: Toolchain Verification Matrix & Sanity Checks

Execute the following verification checklist to ensure all tools are functioning correctly before proceeding to Terraform HCL provisioning:

Tool Name	Verification Command	Expected Version / Output	Status
AWS CLI v2	aws --version	aws-cli/2.15.x or higher	VERIFIED
AWS Auth	aws sts get-caller-identity	Valid AWS Account ID & IAM User ARN	VERIFIED
Terraform CLI	terraform -version	Terraform v1.7.x or higher	VERIFIED
Ansible Core	ansible --version	ansible [core 2.16.x or higher]	VERIFIED
SSH Key Pair	aws ec2 describe-key-pairs	library-aws-key present in AWS	VERIFIED

6. Troubleshooting Common Environment Setup Errors

Symptom / Error Message	Root Cause	Remediation Step
aws: command not found	Binary directory /usr/local/bin is missing from current user PATH environment variable.	Run <code>export PATH=\$PATH:/usr/local/bin</code> and append to <code>~/.bashrc</code> or <code>~/.zshrc</code> .
Signature verification failed (GPG Error)	System CA certificates are outdated or firewall is intercepting HashiCorp GPG key downloads.	Run <code>sudo apt-get install --reinstall ca-certificates</code> and re-fetch key via official URL.
Permissions 0644 for private key are too open	Private SSH key has permissive permissions allowing other users to read it. OpenSSH refuses connection.	Run <code>chmod 400 ~/.ssh/library-aws-key</code> to restrict access exclusively to the file owner.
InvalidAccessKeyId: The AWS Access Key Id does not exist	Typo in Access Key or IAM user was deleted/deactivated in AWS Management Console.	Generate a fresh Access Key pair in the AWS IAM Console and re-run <code>aws configure</code> .

✓ NEXT STEP: INFRASTRUCTURE AS CODE PROVISIONING

With Chapter 2 setup complete, your workstation is ready to proceed to **Chapter 3: Terraform Infrastructure as Code Suite** to provision the AWS VPC, Security Group, EC2 compute host, and S3 backup bucket.